Plan de la «sesión potente» · diagnóstico total + grandes funciones de la app

12/11/2026 · Dirección. **Solo es un plan: no se ha tocado nada.** Está pensado para que una o dos sesiones largas, con muchos agentes trabajando a la vez (Opus o Fable), empiecen y terminen sin depender de nada que no esté aquí. Al final se entrega una **APK usable** y una **lista clara de lo cambiado**.

0. Qué se entrega al final

1. **APK de prueba** «PalomApp prueba» (convive con la de Google Play) y **AAB firmado 1.1.0**, hecho por GitHub (Actions → «App de Android», versión `1.1.0`), listo para subir a la prueba cerrada cuando el dueño lo diga.
2. `docs/CAMBIOS-1.1.0.md` (y en PDF oscuro): por pantallas, con cuatro apartados: **Nuevo · Mejorado · Arreglado · Pendiente**, y al lado de cada línea **«probado en las pruebas automáticas»** o **«sin probar en un móvil real»**.
3. **Informe de diagnóstico** `docs/salud/AAAA-MM-DD-diagnostico.md` + PDF: la lista de problemas encontrada, cada uno con prioridad (P0–P3), cómo reproducirlo, cifra medida y si se arregló en la sesión.
4. **Todo en ramas, nada en** `main` hasta que el dueño diga «publica» (CLAUDE.md). La entrega lleva el nombre de la rama de integración y lo que falta para publicar.
5. Textos para el dueño: novedades para «cerramos día» (sin publicarlas), ficha de Google Play en inglés y mensaje para los probadores.

1. Lo que necesito del dueño ANTES de la sesión (para que sea autosuficiente)

#	Qué	Para qué	Si no está
1	**Capturas de avisos reales** de Santander, Trade Republic, Sabadell y N26: 3–4 de cada (pago con tarjeta, Bizum enviado y recibido, transferencia, ingreso/nómina, devolución). Cifras y nombres tapados	Afinar el lector de avisos con textos de verdad	Se sigue con textos inventados (riesgo de que algún banco no se lea)
2	**Un extracto de cada banco** que uses (CSV, Excel o PDF tal cual lo da la app/web del banco). Se usan solo en el ordenador de la sesión y se borran al terminar; nunca van al repositorio	Importación mejorada: probar formatos reales	Solo se prueban formatos inventados
3	**Proyecto de Firebase** creado (gratis): consola.firebase.google.com → «Añadir proyecto» → añadir app Android `es.palomapp.finanzas` (y `es.palomapp.finanzas.prueba`) → descargar `google-services.json`. Y una «cuenta de servicio» (Configuración → Cuentas de servicio → Generar clave) que se pega en los **secretos de Supabase** como `FCM_CUENTA_SERVICIO` (te guío al empezar si hace falta)	Avisos que llegan aunque la app esté cerrada (Juntos, cobros)	Solo avisos que programa el propio móvil
4	**Tu móvil**: modelo y versión de Android, y si tienes otro Android viejo para probar	Ajustar el rendimiento al móvil real	Se prueba con un móvil «tipo» simulado
5	**Decisiones** de la sección 6 (marcadas «decide el dueño»)	Que nadie tenga que parar a preguntar	Se toma la opción recomendada y se apunta
6	**Permisos de la sesión**, dichos por escrito al empezar: (a) aplicar las migraciones de la lista §5 sin volver a preguntar; (b) lanzar compilaciones en GitHub; (c) que **no** se publique nada	Que la sesión no se quede esperando	Se prepara todo y las migraciones quedan sin aplicar
7	`ORS_KEY` renombrada en Supabase (km del trayecto)	Cerrar ese pendiente	Sigue el aviso de «pon los km a mano»
8	Si se tocan los textos de `auth-signup.js` (entrar/registrarse en inglés): permiso expreso, porque CLAUDE.md prohíbe tocarlo	Que la pantalla de entrada salga en inglés	Esa pantalla se queda en español

2. Fase A · Diagnóstico total (primeras horas)

Cada departamento trabaja en **su propia copia** (`git clone /home/claude/urlife /tmp/<dpto>`), nunca en la carpeta compartida. Se usan **3 juegos de datos inventados**: *ligero* (50 movimientos), *normal* (2.000, 3 cuentas, 2 grupos) y *pesado* (10.000 movimientos, 8 cuentas, 6 grupos, 300 tareas, 2 años). Se generan con un script nuevo `tests/datos/generar.js` (reutiliza `tests/capturas/datos-falsos.js`).

A1 · Rendimiento (Calidad, con Desarrollo)

Medir con Playwright y Chromium **simulando un Android de gama media** (CPU 4× más lenta, red «4G lenta»), con los tres juegos de datos: - ☐ **Arranque:** tiempo hasta ver Inicio con datos (frío y con caché). Separar: descarga, lectura de `app.js` (12.900 líneas),

primera pintura y llegada de datos de Supabase. - ☐ **Coste de `render()` por pestaña** (Inicio, Movimientos Resumen/Lista/Calendario, Juntos, Dinero, Agenda, Tú) y de abrir cada hoja (+, gasto con teclado, filtros, importar, cuenta, grupo). Instrumentar con `performance.mark`. Objetivo: < 50 ms con datos normales, < 150 ms con pesados. - ☐ **Tareas largas** (> 50 ms) al cambiar de pestaña, deslizar el mes, escribir en el teclado del importe y en el buscador. - ☐ **Fotogramas** al abrir/cerrar hojas, deslizar meses (`movDeslizar`) y hacer scroll en Lista con 10.000 movimientos (¿hace falta pintar solo lo visible?). - ☐ **Memoria**: 50 cambios de pestaña y 30 hojas abiertas/cerradas → ¿crece sin parar? (oyentes, temporizadores, suscripciones). - ☐ **Red**: cuántas consultas a Supabase al entrar y al volver a la app, cuántos datos baja cada una, si se piden cosas repetidas, tiempo real (canales abiertos). - ☐ **Peso**: `app.js`, `app.css`, `look.css`, letras y logos; qué se puede cargar más tarde (Informes, Importar, Trayecto, Inversiones). - ☐ **Cálculos repetidos**: `kpisMes`, `cuentasCfg`, saldos, metas, Juntos: cuántas veces se calculan por pintado.

A2 · App de Android (Desarrollo + Calidad, con el móvil del dueño al final)

- ☐ Arranque en frío y al volver tras 1 h en segundo plano (¿se queda en «Cargando…»?).
- ☐ Botón atrás en **todas** las pantallas y hojas (lista completa, con lo esperado en cada una).
- ☐ Teclado del móvil tapando campos (formularios largos, buscador, importar), zonas seguras (muesca, barra de gestos), modo oscuro del sistema.
- ☐ **Letra grande** del sistema (130 % y 200 %): textos cortados o que se pisan.
- ☐ Sin conexión / modo avión: qué pasa al abrir, al apuntar y al volver la red.
- ☐ Enlaces de invitación (`?unirse=`), compartir (Excel, copia, .ics), elegir archivo para importar, cámara del ticket, huella, avisos locales con el móvil en reposo (Doze), consumo de batería del lector de avisos.
- ☐ Versión de WebView mínima que funciona; Android 8 a 16.

A3 · Lógica y cuentas que cuadran (Calidad)

- ☐ **Recorrer tocando** todos los flujos (como `tests/toques_comun.js`): lista de todas las acciones `H.*` y comprobar que cada una se usa y funciona; buscar acciones muertas y botones que no hacen nada.
- ☐ **Las cifras cuadran** en todas partes con los 3 juegos de datos: Inicio = Movimientos = balance del mes = Informes = Dinero («Lo que tienes») = saldos de cuentas; con «Tu mes» del 1 y desde el día de cobro; meses de 28/29/31 días; cambio de año.
- ☐ Juntos: saldos de grupo = «¿De dónde sale?» = hoja de cada persona, con monedas de viaje, personas sin cuenta y «Deshacer».
- ☐ Importar: el mismo extracto dos veces no duplica; con avisos del banco ya apuntados no duplica (`origen: 'aviso'`).
- ☐ Redondeos (céntimos) y formatos («1.234,56», «12,5», negativos).
- ☐ Errores en consola durante toda la batería (cero permitido).

A4 · Interfaz y móvil (Producto y Diseño + Calidad)

- ☐ Capturas automáticas (`tests/capturas/revisar.js`) ampliadas a **320, 360, 390 y 412 px**, tableta 768 px en horizontal, claro y oscuro, las 5 paletas y letra al 130 %.
- ☐ Zonas táctiles < 44 px, contraste por debajo de AA, textos cortados o que se salen, iconos sin nombre accesible.
- ☐ Repasar lo pendiente de `docs/11` (lo «C») y `docs/32` (B4, B6, C3, C5) y decir qué sigue pasando.
- ☐ Coherencia: mismas palabras para lo mismo, mismas cabeceras de periodo, mismos botones de guardar/cancelar, mismos vacíos.

A5 · Datos y seguridad (Datos y Seguridad)

- ☐ Avisos de seguridad y rendimiento de Supabase (`get_advisors`); permisos (RLS) de **todas** las tablas y funciones; índices que faltan para las consultas reales.
- ☐ Funciones (`tickets-ia`, `trayecto`, `activos-api`, `borrar-cuenta`): límites, errores, coste al mes.
- ☐ Repositorio público: ni claves ni datos personales (también en `docs/` e imágenes).
- ☐ Altas: correo y CAPTCHA; límite de gasto en Anthropic; copia de seguridad diaria (pendiente desde octubre).
- ☐ Qué datos guarda el móvil (localStorage, cola de avisos) y que se borran al cerrar sesión o borrar la cuenta.

A6 · Herramientas (Desarrollo + Calidad)

- ☐ Batería de pruebas: hoy tarda ~15 min. Ejecutarla en paralelo y separar «rápidas» (< 3 min) de «completas».
- ☐ **GitHub Actions**: que cada rama pase las pruebas web + las del lector de Android al subir.
- ☐ Revisión automática del código (errores tontos) y del peso de los archivos.
- ☐ Aviso de errores de la app en producción: qué opciones baratas hay y qué cuestan.

Salida de la fase A: `docs/salud/…-diagnostico.md` con la tabla *Problema · Dónde · Cómo se reproduce · Medida · Prioridad (P0 rompe/pierde datos, P1 molesta mucho, P2 mejora, P3 detalle) · Dueño (departamento).* Los **P0 y P1 se arreglan en la misma sesión** (fase B) antes de las funciones nuevas.

3. Fase B · Arreglar lo del diagnóstico

- Desarrollo arregla P0 y P1 en la rama `fase13/arreglos`, cada uno con su prueba. Calidad lo comprueba con ojos frescos.
- Mejoras de rendimiento con más efecto (a confirmar con las medidas): pintar solo lo visible en listas largas, no recalcular lo que no cambia, cargar tarde las pantallas poco usadas, repintar solo la parte que cambia.

4. Fase C · Las grandes funciones (cada una: Producto → Datos → Desarrollo → Calidad)

C1 · Idiomas (español e inglés; preparado para más)

- **Cómo:** un diccionario `i18n/es.js` + `i18n/en.js` y una función `t('clave')`; fechas, números y moneda con el idioma (`Intl`). El idioma sale del móvil la primera vez y se cambia en Tú → La app → «Idioma». Se guarda en `cfg().idioma`.
- **Qué se traduce:** todos los textos de la app, avisos, recordatorios, novedades, privacidad, borrar cuenta, la ficha de Google Play y las categorías **de serie** (las que crea el usuario no se tocan).
- **Ojo:** la pantalla de entrada/registro está en `auth-signup.js` (no se toca sin permiso, punto 8 de §1). Los nombres de bancos y las palabras de la app («Aportar», «Juntos») se deciden en inglés con el dueño.
- **Pruebas:** toda la batería pasa en los dos idiomas; un «idioma falso» que marca lo que se quedó sin traducir; capturas en inglés para ver textos más largos.
- **Hecho cuando:** ningún texto en español con la app en inglés (salvo datos del usuario), y las cifras salen como «€1,234.56».

C2 · Avisos de la app (estructura pensada para el usuario)

Nivel	Cuándo suena	Ejemplos	Ajuste por defecto
Importantes (sonido, se ven en pantalla)	Algo que pide hacer algo hoy	Alguien te pide dinero o te ha saldado; un pago fijo hoy y no hay saldo en esa cuenta; te han invitado a un grupo	Encendido
Normales (sin sonido, en la barra)	Para estar al día	Pago o cobro fijo mañana; tareas de hoy; gasto nuevo en tu grupo; presupuesto al 80 % y al 100 %; meta conseguida; movimientos apuntados de tus avisos	Encendido
Resumen (uno a la semana o al mes)	Cuando hay calma	Resumen del domingo; tu mes cerrado; informe listo	Apagado (se ofrece)

Reglas: - Nada de 22:00 a 9:00 (se puede cambiar). - Como mucho 3 al día, salvo los importantes. Los de un mismo tipo se agrupan («3 gastos nuevos en Piso»). - Sin cifras con el móvil bloqueado. - Tocar el aviso lleva a la pantalla justa. - Un interruptor por tipo en Tú → Avisos. - Nunca avisos de marketing.

Cómo: - Lo que se sabe en el móvil (fijos, Agenda, presupuestos, metas) se programa en el propio móvil (ya hay base). - Lo que hacen otros (Juntos) llega por Firebase: - tabla `avisos_dispositivos` (id del móvil, token, idioma); - función de Supabase `avisar` que se dispara al crear un gasto compartido o un pago; - preferencias en `cfg().avisos`.

Hecho cuando Ana apunta un gasto en el grupo y a Álvaro le llega un aviso en menos de un minuto con la app cerrada, y lo puede apagar.

C3 · Leer los avisos del banco · fase 2

- **Lector:** afinarlo con las capturas reales (punto 1 de §1). Cada aviso real se convierte en una prueba con el texto inventado equivalente; nunca se guarda el texto real.
- **Guardar con la app cerrada:** desde Android, a una función de Supabase con una clave propia de ese móvil, que se puede retirar (tabla `avisos_dispositivos`, la misma de C2).
- **«Deshacer» desde el propio aviso:** se puede borrar sin abrir la app.
- **Contador sin datos** (cuántos leídos, apuntados y borrados) para saber si funciona.
- **Tarjeta en «Descubre»** y textos para la ficha de Google Play y la «Seguridad de los datos». Se sube a Google Play cuando haya acceso a producción.

C4 · Importación de extractos mejorada

- **Diagnóstico de lo que hay:** el importador actual (funciones `imp*` de `app.js`) contra los extractos reales del punto 2. Para cada banco: ¿lo reconoce?, ¿columnas bien?, ¿signo?, ¿fechas?
- **Formatos:** CSV y Excel de cada banco, **Norma 43 (AEB 43)**, que es el formato estándar de los bancos españoles, y PDF si el banco solo da PDF (estudiar si merece la pena).
- **«Compartir con PalomApp»:** desde la app del banco o de Archivos, el extracto se abre directamente en Importar (filtro de archivos en Android).
- **Recordar** el formato y la cuenta de cada banco, para que la segunda vez sea un toque.

- **No duplicar** con lo apuntado a mano ni con lo que vino de los avisos.
- **Hecho cuando** el extracto de cada banco del dueño entra bien a la primera, y el mismo extracto dos veces no duplica nada.

C5 · Widgets de Android

1. **«Te queda este mes»** (2×1): la cifra y una barra. Si están ocultas las cifras, sale «•••».
2. **«Apuntar gasto»** (1×1): abre directamente el gasto con el teclado.
3. **Opcional, «Hoy»** (2×2): tareas de hoy y el próximo pago fijo.
4. **Cómo:** la app deja un resumen pequeño en el móvil cada vez que cambia algo, y el widget lo lee. Se actualiza al abrir la app y cada pocas horas.
5. **Hecho cuando** el widget enseña lo mismo que Inicio y apuntar desde él tarda 2 toques.

C6 · Otras (por orden, si da tiempo)

1. **Funcionar sin conexión:** ver lo último que se cargó y poder apuntar sin red; se sube al volver la conexión (`docs/32` C3).
2. **Resto de pendientes:** B4 y B6 de `docs/32` y las piezas de 44 px (C5 de `docs/32`).
3. **iPhone:** solo estudiar qué haría falta. No se programa.

---

5. Migraciones previstas (solo aditivas; con el «sí» del dueño al empezar)

- `avisos_dispositivos` : un móvil por fila, con id, usuario, token de Firebase, plataforma, idioma, fecha y si está activo. Solo lo ve su dueño (RLS).
- Opcional, `avisos_enviados` : registro mínimo para no repetir avisos y respetar el «máximo 3 al día». Se borra a los 30 días.
- Funciones nuevas: `avisar` (Firebase) y `avisos-guardar` (avisos del banco con la app cerrada). Ningún cambio en tablas existentes ni en datos.

---

6. Decisiones para el dueño (si no contesta, se toma la recomendada)

Pregunta	Recomendado
¿Qué idiomas?	Español e inglés ahora, preparado para más
¿Cómo se dicen en inglés «Juntos», «Aportar» y «Tu mes»?	«Together», «Add money», «Your month»
¿Quién revisa las traducciones?	Las hace la sesión, y un amigo que hable inglés les da un repaso rápido antes de publicar
¿Avisos de «Resumen» encendidos de serie?	No: se ofrecen al cabo de 2 semanas
¿Horas sin avisos?	De 22:00 a 9:00
¿Qué widgets?	«Te queda este mes» y «Apuntar gasto»
¿Importar PDF?	Solo si algún banco tuyo no da ni CSV ni Excel
¿Subir 1.1.0 a la prueba cerrada al terminar?	Sí, pero sin los avisos del banco hasta tener producción

---

7. Cómo se organiza la sesión (para que vaya rápido y sin pisarse)

- **Ramas:**
- `fase13/diagnostico` (solo informes y pruebas);
- `fase13/arreglos` ;
- una por función: `fase13/idiomas` , `fase13/avisos-app` , `fase13/avisos-banco-2` , `fase13/importar` , `fase13/widgets` ;
- todas se juntan en **`fase13/integracion`** , que es de donde sale la APK.
- **Cada agente en su copia** (`/tmp/<rama>`), nunca en la carpeta compartida (ya hubo líos con `git checkout` a la vez).
- **Menos choques en `app.js` :** lo nuevo va en archivos aparte cuando se pueda (`i18n/*.js` , `avisos.js` , `widgets` en Android), cargados desde `index.html` y copiados por `movil/scripts/copiar-web.js` . **Idiomas es lo que más toca**: va primero o al final, nunca a la vez que otra función que cambie textos.
- **Orden:**
- A (diagnóstico, todos a la vez);
- B (arreglos);
- C2, C3, C4 y C5 a la vez (casi no se tocan entre ellas);
- C1 (idiomas, cuando lo demás está quieto);
- integración;

- Calidad completa;
- APK y AAB;
- lista de cambios.
- **Puntos de control:** al acabar cada fase, un resumen de 5 líneas al dueño. Si algo P0 no se puede arreglar, se para y se avisa.
- **Lo que hay en el ordenador de la sesión:** Android SDK, Gradle y Java (compilan la APK y las pruebas del lector), Playwright con Chromium (capturas y medidas), la conexión con Supabase y GitHub (Actions firma el AAB con los secretos ya guardados).
- **Lo que no hay:** emulador de Android ni móvil real. Lo de Android se prueba al final en el móvil del dueño con la APK.

8. Riesgos

- **Tamaño de la tarea:** demasiadas cosas a la vez. Prioridad si falta tiempo: arreglos P0/P1 → avisos de la app → widgets → importar → avisos del banco 2 → idiomas.
- **Idiomas:** cambian miles de textos y muchas pruebas que comprueban textos. Por eso va al final y con «idioma falso».
- **Firebase:** sin el proyecto creado (punto 3), C2 se queda en avisos locales.
- **Google Play:** subir avisos del banco o el permiso de notificaciones puede pedir revisión. Se suben sin ellos si hace falta.
- **Datos reales del dueño** (capturas y extractos): solo en el ordenador de la sesión, nunca en el repositorio; se borran al terminar.

9. Texto para empezar la sesión (copiar y pegar)

> Empezamos la sesión potente de `docs/40-plan-sesion-potente.md`. Sigue ese plan de principio a fin sin pararte a preguntar: si algo no está decidido, toma la opción recomendada y apúntala. Te doy permiso para: aplicar las migraciones de §5, compilar en GitHub y usar todos los agentes que hagan falta, cada uno en su copia. No publiques nada en `main`. [Adjunto: capturas de avisos, extractos, `google-services.json` / «Firebase pendiente», mi móvil es …]. Al terminar quiero la APK de prueba, el AAB 1.1.0, `docs/CAMBIOS-1.1.0.md` en PDF y el informe de diagnóstico en PDF.